CO1-AT3-Coding Chellenge

View Only

← Back

QUESTIONS

Q1
Smart University Payroll Management System
25 marks

Q2
Smart Vehicle Insurance Claim System
25 marks

2/2 answered

Q1 Smart University Payroll Management System

25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)
Input
3

TeachingFaculty
101
Dr. Arun
CSE
50000
20
500

TechnicalStaff
102
Mr. Ravi
ECE
35000
15
300

VisitingFaculty
103
Dr. Kumar
IT
0

40
1200

Search Employee ID:
102
Expected Output
Employee Details

Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0

Employee ID : 102
Name : Mr. Ravi
Department : ECE
Monthly Salary : 39500.0

Employee ID : 103
Name : Dr. Kumar
Department : IT
Monthly Salary : 48000.0

Highest Paid Employee
Employee ID : 101
Salary : 60000.0

Average Salary
49166.67

Employee Found
Employee ID : 102
Name : Mr. Ravi
Salary : 39500.0

Your Code

```java
import java.util.Scanner;

class Employee {
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;
    public Employee() {}
    public Employee(int employeeId, String employeeName, String depart
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
```

Input

```
3
TeachingFaculty
101
Dr. Arun
```

```
        this.basicSalary = basicSalary;
    }
    public int getEmployeeId() { return employeeId; }
    public String getEmployeeName() { return employeeName; }
    public String getDepartment() { return department; }
    public double getBasicSalary() { return basicSalary; }
    public void setEmployeeId(int employeeId) { this.employeeId = empl
    public void setEmployeeName(String employeeName) { this.employeeNa
    public void setDepartment(String department) { this.department = d
    public void setBasicSalary(double basicSalary) { this.basicSalary
    public double calculateSalary() {
        return basicSalary;
    }
    public void displayEmployee() {
        System.out.println("Employee ID : " + employeeId);
        System.out.println("Name : " + employeeName);
        System.out.println("Department : " + department);
        System.out.println("Monthly Salary : " + calculateSalary());
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(readNonEmptyLine(sc));
        Employee[] employees = new Employee[n];
        for (int i = 0; i < n; i++) {
            String type = readNonEmptyLine(sc);
            int id = Integer.parseInt(readNonEmptyLine(sc));
            String name = readNonEmptyLine(sc);
            String dept = readNonEmptyLine(sc);
            double basicSalary = Double.parseDouble(readNonEmptyLine(s
            if (type.equals("TeachingFaculty")) {
                double teachingHours = Double.parseDouble(readNonEmpty
                double incentivePerHour = Double.parseDouble(readNonEm
                employees[i] = new TeachingFaculty(id, name, dept, bas
            } else if (type.equals("TechnicalStaff")) {
                double overtimeHours = Double.parseDouble(readNonEmpty
                double overtimeRate = Double.parseDouble(readNonEmptyL
                employees[i] = new TechnicalStaff(id, name, dept, basi
            } else if (type.equals("VisitingFaculty")) {
                double lecturesHandled = Double.parseDouble(readNonEmp
                double paymentPerLecture = Double.parseDouble(readNonE
                employees[i] = new VisitingFaculty(id, name, dept, bas
            }
        }
        System.out.println("Employee Details");
        for (Employee e : employees) {
            System.out.println();
            e.displayEmployee();
        }
        System.out.println();
        Employee highestPaid = employees[0];
        for (Employee e : employees) {
```

Output

```
Employee Details

Employee ID : 101
Name : Dr. Arun
Department : CSE
Monthly Salary : 60000.0
```

```
            if (e.calculateSalary() > highestPaid.calculateSalary()) {
                highestPaid = e;
            }
        }
        System.out.println("Highest Paid Employee");
        System.out.println("Employee ID : " + highestPaid.getEmployeeI
        System.out.println("Salary : " + highestPaid.calculateSalary()
        System.out.println();
        double total = 0;
        for (Employee e : employees) total += e.calculateSalary();
        double average = total / n;
        System.out.println("Average Salary");
        System.out.printf("%.2f%n", average);
        System.out.println();
        String line = readNonEmptyLine(sc);
        int searchId;
        if (line.contains("Search Employee ID")) {
            searchId = Integer.parseInt(readNonEmptyLine(sc));
        } else {
            searchId = Integer.parseInt(line);
        }
        boolean found = false;
        for (Employee e : employees) {
            if (e.getEmployeeId() == searchId) {
                System.out.println("Employee Found");
                System.out.println("Employee ID : " + e.getEmployeeId(
                System.out.println("Name : " + e.getEmployeeName());
                System.out.println("Salary : " + e.calculateSalary());
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Employee Not Found");
        }

        sc.close();
    }
    private static String readNonEmptyLine(Scanner sc) {
        while (sc.hasNextLine()) {
            String s = sc.nextLine().trim();
            if (!s.isEmpty()) return s;
        }
        return "";
    }
}

class TeachingFaculty extends Employee {
    private double teachingHours;
    private double incentivePerHour;
    public TeachingFaculty(int employeeId, String employeeName, String
```

```
                            double basicSalary, double teachingHours, d
        super(employeeId, employeeName, department, basicSalary);
        this.teachingHours = teachingHours;
        this.incentivePerHour = incentivePerHour;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (teachingHours * incentivePerHour);
    }
}

class TechnicalStaff extends Employee {
    private double overtimeHours;
    private double overtimeRate;
    public TechnicalStaff(int employeeId, String employeeName, String
                          double basicSalary, double overtimeHours, do
        super(employeeId, employeeName, department, basicSalary);
        this.overtimeHours = overtimeHours;
        this.overtimeRate = overtimeRate;
    }
    @Override
    public double calculateSalary() {
        return getBasicSalary() + (overtimeHours * overtimeRate);
    }
}
class VisitingFaculty extends Employee {
    private double lecturesHandled;
    private double paymentPerLecture;
    public VisitingFaculty(int employeeId, String employeeName, String
                           double basicSalary, double lecturesHandled,
        super(employeeId, employeeName, department, basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }
    @Override
    public double calculateSalary() {
        return getBasicSalary() + (lecturesHandled * paymentPerLecture
    }
}
```

Submitted